

ひまわり予定表 技術設計解説書

コンポーネント・関数・Props・フック・API・データベース設計を
すべて分解し、使われている技術と設計思想を網羅的に解説する

プロジェクト	schedule-next
概要	療育施設の日程を管理する Web アプリケーション
スタック	Next.js 16 / React 19 / TypeScript / Prisma 7 / MySQL
作成日	2026年8月6日

本書はソースコード全ファイルを読み取り、実装に基づいて作成されています。

目次

01 全体像 — このアプリは何層でできているか
レイヤードアーキテクチャ／2本のサーバー経路

02 技術スタック一覧
各ライブラリが何を解決しているか

03 データベース設計
Child / Attendance / 認証テーブル / 制約とインデックス

04 Prisma のセットアップ
シングルトン・Driver Adapter・fail-fast

05 サーバロジック層
Repository パターン / Service 層 / DTO

06 日付ロジック
getWeeks の完全分解・タイムゾーン設計

07 バリデーション層 (Zod)
スキーマファースト・型の自動導出

08 認証とアクセス制御
3重の防御・401と403・database 戦略

09 API ルート (Route Handler)
処理順序の設計・構造化ログ

10 クライアント層
Server/Client 境界・React Query・React Hook Form

11 型の流れ
3つの源流と、それらを分けている理由

12 テスト
純粋関数から書く理由・テストケースの選び方

13 ツール・設定まわり
tsconfig / ESLint / Prettier / Prisma Config

14 未完成部分・改善点
現状の課題と次の実装ステップ

15 まとめ
この設計を一言で説明するなら

01 全体像 — このアプリは何層でできているか

療育施設の予定表アプリです。データの流れは一方向で、責務ごとに層が分かれています。まずこの地図を頭に入れると、以降の各章がどこの話なのか迷わなくなります。



1-1. レイヤーアーキテクチャ

これはレイヤーアーキテクチャ（3層 + プレゼンテーション層）です。発想の核は「各層は自分の下の層しか知らない」ことにあります。

層	知っていること	知らないこと
Route	HTTP、ステータスコード、セッション	SQL、Prisma のクエリ構文
Service	業務ルール（ページング計算、DTO 整形）	HTTP、Request / Response オブジェクト
Repository	Prisma のクエリ	業務ルール、HTTP

なぜ分けるのか

テストしやすさと差し替えやすさのためです。Service は `Request` を受け取らないので、テストで HTTP を用意しなくても直接呼べます。DB を PostgreSQL に変えても Repository だけ直せば済みます。逆に「全部 route.ts に書く」と、テストのたびに HTTP と DB の両方を用意する必要が出てきます。

1-2. サーバーへの道が2本ある

App Router 特有の重要な点として、このアプリにはサーバーのデータに到達する経路が2本あります。

経路	通り道	使っている箇所
Server Component 経由	HTTP を通らず、サーバー上で直接関数を呼ぶ	<code>page.tsx</code> の <code>auth()</code>
API Route 経由	ブラウザから <code>fetch</code> で HTTP リクエスト	<code>/api/children</code> 、 <code>/api/attendances</code>

「サーバーで直接読める」のに、なぜ API も用意しているのか。理由は**更新後の再取得**です。児童を追加した直後に一覧を更新したいとき、Server Component だとページ全体の再読み込みが必要になります。API + React Query なら、一覧部分だけを裏で差し替えられます。「初回表示はサーバー、以降の更新はクライアント」という使い分けです。

02 技術スタック一覧

それぞれのライブラリが「どんな問題を解決するために入っているのか」を押さえます。名前を挙げるだけでなく、なぜ必要かを言えることが重要です。

技術	分類	解決している問題
Next.js 16	フレームワーク	App Router によるサーバー／クライアント統合、ルーティング、ビルド最適化
React 19	UI ライブラリ	宣言的UI。Server Components と Server Actions を提供
TypeScript	言語	ビルド時の型検査。 <code>strict: true</code> で null 安全も担保
Prisma 7	ORM	スキーマからの型生成、マイグレーション管理、SQL インジェクション対策
@prisma/adapters-mariadb	DB ドライバ	Rust エンジン不要の JS ドライバ接続（Driver Adapter 方式）
Auth.js (next-auth v5)	認証	GitHub OAuth、セッション管理。パスワードを自前で持たない
@auth/prisma-adapter	認証	Auth.js のセッション保存先を Prisma に接続するアダプター
Zod 4	検証	実行時バリデーション＋型の自動導出。外部入力の安全化
TanStack Query 5	状態管理	サーバー状態のキャッシュ、再取得、ローディング／エラー管理
React Hook Form 7	フォーム	非制御コンポーネントによる高速なフォーム。再レンダリング最小化
@hookform/resolvers	接続	React Hook Form と Zod をつなぐアダプター
Sonner	UI	トースト通知。成功／失敗のフィードバック
Tailwind CSS 4	スタイル	ユーティリティクラスによるスタイリング。未使用CSSの自動削除
Vitest 4	テスト	Vite ベースの高速なユニットテスト実行環境
Prettier / ESLint	品質	フォーマット統一と静的解析

構成の特徴

「アダプター」が3つ入っているのがこの構成の性格をよく表しています。`PrismaAdapter`（Auth.js ↔ Prisma）、`PrismaMariaDb`（Prisma ↔ MariaDB）、`zodResolver`（RHF ↔ Zod）。それぞれのライブラリが特定の相手に依存せず、インターフェースだけを決めて実装を差し込める設計になっているため、後から差し替えが効きます。

03 データベース設計

ファイル：`prisma/schema.prisma`。ここでの設計判断は、後から変更するコストが最も高い部分です。制約をどこまで DB に持たせるかが焦点になります。

3-1. テーブル一覧

モデル	役割	出所
<code>Child</code>	児童のマスタデータ	自作
<code>Attendance</code>	「誰が・いつ・どう来るか」の予定	自作
<code>TimeFrame</code> (enum)	午前 / 午後	自作
<code>User</code> / <code>Account</code> / <code>Session</code> / <code>VerificationToken</code>	認証	Auth.js 標準スキーマ

3-2. Child モデル

```
model Child {
  id          Int           @id @default(autoincrement())
  name        String
  contractDays Int          @default(0)
  color       String
  sortOrder   Int           @default(0)
  createdAt   DateTime      @default(now())
  updatedAt   DateTime      @updatedAt
  defaultTimeFrame TimeFrame?
  attendances Attendance[]
}
```

▶ `sortOrder` — 並び順をユーザーが決める

「並び順をユーザー側で制御できるようにする」ための定番パターンです。`id` 順や `name` 順では現場の都合（学年順、クラス順など）に合いません。整数で持たせ、`orderBy: { sortOrder: "asc" }` でソートします。

▶ `color` — DB に見た目を持たせる判断

カレンダー上で児童を色で識別するため。一見「見た目の情報を DB に置くのはおかしい」と思えますが、これはその児童固有の属性なので妥当です。「テーマカラー」のようなアプリ全体の設定とは性質が違います。

▶ createdAt / updatedAt — 監査カラム

監査カラム（audit columns）と呼ばれる定番です。`@default(now())` は挿入時に、`@updatedAt` は Prisma が更新時に自動でセットします。SQL のトリガーを書く必要がありません。

▶ defaultTimeFrame TimeFrame? — nullable

`?` が nullable を意味します。「この子はいつも午前」というデフォルトを持たせ、`Attendance` 作成時の初期値に使う意図です。

▶ attendances Attendance[] — 実在しないカラム

これは**実際のカラムではありません**。Prisma のリレーションフィールドで、`include: { attendances: true }` と書けるようにするための「仮想フィールド」です。DB のテーブルには存在しません。

3-3. Attendance モデル — 設計上いちばん見どころのある部分

```
model Attendance {
  id          Int          @id @default(autoincrement())
  date        DateTime     @db.Date
  childId     Int
  child       Child        @relation(fields: [childId], references: [id], onDelete: Cascade)
  timeFrame   TimeFrame?
  pickup      Boolean       @default(false)
  dropoff     Boolean       @default(false)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  @@unique([date, childId])
  @@index([date])
}
```

▶ @db.Date — 型でドメインを表現する

Prisma のデフォルトは `DATETIME`（時刻付き）ですが、`@db.Date` で MySQL の `DATE` 型（時刻なし）にしています。「**予定は日付単位であって時刻ではない**」というドメインの性質を、型で表現しています。これによりタイムゾーンによる時刻ずれの余地が減ります（完全には消えません。第6章で詳述）。

▶ @@unique([date, childId]) — 複合ユニーク制約

「1人の児童は1日につき1レコードしか持てない」というビジネスルールを、DB レベルで強制しています。

なぜアプリ側のチェックでは不十分か

アプリ側で `if（既にあるか?）` とチェックしても、2つのリクエストが同時に来ると両方すり抜けます（レースコンディション）。チェックと挿入の間に、別のリクエストが挿入してしまうためです。DB のユニーク制約は原子的なので、すり抜けられません。

さらにこの制約は `upsert` の鍵にもなります。

```
prisma.attendance.upsert({
  where: { date_childId: { date, childId } }, // ← @@unique が生成した複合キー名
  create: { ... },
  update: { ... },
})
```

「チェックボックスを押したら、なければ作る・あれば更新する」という予定表アプリに必須の操作が、**1クエリ**で書けます。SELECT してから分岐する必要がありません。

▶ @@index([date]) — 範囲検索の高速化

`attendanceRepository.ts` の `where: { date: { gte: start, lt: end } }` という**範囲検索を高速化**するためのインデックスです。これがないと全行スキャン（フルテーブルスキャン）になります。B-Tree インデックスは範囲検索が得意なので、月単位の取得にぴったりです。

▶ onDelete: Cascade — 参照整合性を DB に任せる

児童を削除したら、その子の予定も自動で消えます。アプリ側で「先に attendance を消してから child を消す」という手順を書かなくて済み、書き忘れによる**孤児レコード**（orphan record）も発生しません。

▶ pickup / dropoff を Boolean 2本に分けた理由

`transportType: "PICKUP" | "DROPOFF" | "BOTH" | "NONE"` という enum にもできますが、Boolean 2本のほうが**UI のチェックボックス2つと 1:1 で対応し、組み合わせが増えても破綻しません。UI の操作単位とデータモデルを一致させる判断**です。

3-4. 認証テーブル（Auth.js 標準スキーマ）

`User` / `Account` / `Session` / `VerificationToken` は、Auth.js の Prisma Adapter が要求する**固定スキーマ**です。自分で設計したものではなく「規約に合わせた」ものです。

▶ Account の複合主キー

```
@@id([provider, providerAccountId])
```

「GitHub の ID 12345」という組み合わせで一意になります。この設計により、**1人の User が GitHub と Google の両方を紐付けられます**（アカウントリンクング）。

▶ トークン列が @db.Text である理由

`refresh_token` / `access_token` / `id_token` はすべて `@db.Text` です。OAuth のトークン、特に JWT である `id_token` は数百～数千文字になります。MySQL における Prisma のデフォルト `VARCHAR(191)` では入りきらないため、`TEXT` に拡張しています。Auth.js のドキュメントに明記されている**必須対応**です。

▶ **ここだけ snake_case である理由**

他が camelCase なのに `refresh_token` だけスネークケースなのは、OAuth 2.0 の仕様書のフィールド名をそのまま使っているためです。Auth.js の規約であり、変更できません。

▶ **session: { strategy: "database" } — 戦略の選択**

	JWT 戦略	database 戦略（採用）
保存場所	Cookie の中（署名付き）	DB の <code>Session</code> テーブル
DB アクセス	不要（速い）	毎回必要
即時失効	できない（有効期限まで有効）	DB の行を消せば即ログアウト
スケール	得意（ステートレス）	DB が単一障害点になりうる

なぜ database 戦略が正しいか

このアプリは `approved` フラグによる承認制です。「承認を取り消したら即座に使えなくする」ことが要件になります。JWT 戦略だとトークンの有効期限が切れるまで使い続けられてしまうため、database 戦略が必然的な選択になります。

▶ **User.approved — 標準スキーマへの独自拡張**

`approved Boolean @default(false)`

Auth.js の標準スキーマに自分で追加したフィールドです。GitHub でログインできても、管理者が `approved = true` にするまで使えません。招待制／ホワイトリスト方式であり、療育施設という個人情報を扱うアプリとして妥当な設計です。

3-5. マイグレーション履歴

```
20260802163315_add_child_table
20260803044839_add_auth_tables
20260806064057_add_attendance
20260806064652_npx_prisma_generate
```

マイグレーション履歴によるスキーマのバージョン管理です。タイムスタンプ順に SQL が積み上がり、`prisma migrate deploy` で本番環境に同じ順序で適用されます。Git のコミット履歴と同じ発想で、スキーマの変更が再現可能になります。「開発環境と本番でテーブル定義が違う」という事故を防ぎます。

04 Prisma のセットアップ

ファイル: `src/lib/prisma.ts`。わずか十数行ですが、5つの技術要素が詰まった Next.js × Prisma の定番イディオムです。

```
const connectionString = process.env.DATABASE_URL;
if (!connectionString) {
  throw new Error("DATABASE_URL is not set");
}

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({ adapter: new PrismaMariaDb(connectionString) });

if (process.env.NODE_ENV !== "production") {
  globalForPrisma.prisma = prisma;
}
```

4-1. シングルトンパターン

`PrismaClient` は内部にコネクションプールを持ちます。インスタンスを何個も作ると DB への接続数が爆発し、`Too many connections` エラーになります。だからアプリ全体で 1 つだけにします。

4-2. globalThis へのキャッシュ

開発中の HMR (Hot Module Replacement) は、ファイルを保存するたびにモジュールを再評価します。普通にモジュールスコープで `new PrismaClient()` すると、保存のたびに新しいクライアントが増え続けます。

`globalThis` は HMR でもリセットされないため、そこに退避させます。`if (NODE_ENV !== "production")` で本番を除外しているのは、本番には HMR がなく、グローバル汚染を避けたいからです。

4-3. as unknown as — 二段キャスト

`globalThis` の型に `prisma` プロパティは存在しないので、TypeScript は直接のキャストを拒否します。一度 `unknown` を経由することで強制的に型を付けるエスケープハッチです。乱用は禁物ですが、この用途では標準的な書き方です。

4-4. ?? — Null 合体演算子

`||` ではなく `??` を使うのは、`||` だと `0` や `""` も右辺に流れてしまうためです。「null / undefined のときだけ」という意図を正確に表現します。

4-5. Driver Adapter

Prisma 6 まではネイティブの Rust エンジンが DB に接続していました。Prisma 7 では **Driver Adapter** 方式が主流で、JS のドライバ（ここでは MariaDB 用）を差し込みます。Rust バイナリが不要になり、Edge ランタイムやサーバーレス環境で動かしやすくなります。

4-6. fail-fast — 起動時に落とす

```
if (!connectionString) throw new Error("DATABASE_URL is not set");
```

環境変数がなければ**モジュール読み込み時点で落とします**。リクエストが来たら「undefined に接続できません」という謎のエラーが出るより、起動時に明確なメッセージで落ちるほうが、原因特定が圧倒的に速くなります。これを **fail-fast** と呼びます。

05 サーバロジック層

5-1. Repository 層

ファイル: `src/repositories/`

```
// childRepository.ts
export function createChild(data: Prisma.ChildCreateInput) {
  return prisma.child.create({ data });
}
export function findChildren({ skip, take }: { skip: number; take: number }) {
  return prisma.child.findMany({ skip, take, orderBy: { sortOrder: "asc" } });
}
export function countChildren() {
  return prisma.child.count();
}
```

▶ Repository パターン

「DB アクセスをここに閉じ込める」という発想です。Service 層に `prisma.child.findMany(...)` を直接書かないことで、次の利点が得られます。

- SQL / Prisma の知識が 1 ファイルに集約される
- テストで Repository をモックすれば、Service を DB なしでテストできる
- ORM を差し替えても Service は無傷
- 「この操作、どこで使われている？」が追やすい

▶ async / await を書いていない

```
export function createChild(...) {          // async ではない
  return prisma.child.create({ data });    // Promise をそのまま返す
}
```

`await` して `return` するのではなく、**Promise をそのまま返しています**（pass-through）。`async function` にすると Promise を余計に1段包む処理が入るだけで、意味は同じです。中で `await` の結果を加工しないなら、この書き方が簡潔です。

▶ オブジェクト引数（名前付き引数）

`findChildren({ skip, take })` のようにオブジェクトで受け取っています。`findChildren(0, 20)` だと呼び出し側で「どっちが skip だっけ」となります。オブジェクトなら**名前付き引数**として読め、引数の順番を間違えません。引数が2個以上ならこちらが推奨されます。

▶ Prisma.ChildCreatelInput の利用

自分で `{ name: string; color: string; ... }` と型を書かず、Prisma が生成した型を使っています。スキーマを変えれば型も自動で追従するので、型定義の二重管理がなくなります。

5-2. Service 層

ファイル: `src/services/childService.ts`

```
export async function getChildList({ page, limit }) {
  const skip = (page - 1) * limit;
  const [childList, total] = await Promise.all([
    findChildren({ skip, take: limit }),
    countChildren(),
  ]);

  const data = childList.map((child) => ({
    id: child.id, name: child.name, color: child.color,
    contractDays: child.contractDays, sortOrder: child.sortOrder,
  }));

  return { data, page, limit, total, totalPages: Math.ceil(total / limit) };
}
```

この短い関数に4つの技術が入っています。

▶ (1) オフセットページネーション

`skip = (page - 1) * limit` という式が本体です。`page=1` なら skip 0、`page=3, limit=20` なら skip 40。SQL の `LIMIT 20 OFFSET 40` に対応します。

方式	長所	短所
オフセット（採用）	実装が簡単。「3ページ目に飛ぶ」ができる	深いページ（OFFSET 100000）は DB が読み飛ばす行が増えて遅い
カーソル	何ページ目でも高速	「N ページ目にジャンプ」ができない

児童数が数十～数百のこのアプリなら、オフセットで十分という判断です。

▶ (2) Promise.all による並列化

```
const [childList, total] = await Promise.all([findChildren(...), countChildren()]);
```

「一覧取得」と「総件数カウント」は互いに依存しないので同時に投げます。順番に await すると $50\text{ms} + 30\text{ms} = 80\text{ms}$ 、並列なら $\max(50, 30) = 50\text{ms}$ です。

await を2行並べた場合との違い

```
const a = await findChildren(...); // 完了を待ってから
const b = await countChildren();    // ようやく開始 ← 直列になる
```

▶ (3) DTO への変換 — 明示的なフィールド選択

Prisma の返り値には `createdAt` / `updatedAt` も含まれますが、**明示的に必要なフィールドだけ詰め替えて返しています**。これを DTO（Data Transfer Object）と呼びます。目的は3つです。

- **情報漏洩の防止** — DB カラムを増やしたときに、意図せず API から漏れるのを防ぐ。今は無害でも、将来 `Child` に `medicalNote` を足したら自動的に外に出てしまう
- **API 契約の安定** — DB スキーマが変わっても API のレスポンス形が変わらない
- **型の明示** — `types/api.ts` の `Child` 型と 1:1 で対応する

これは意識的な設計判断です

「Prisma の結果をそのまま `Response.json()` する」のが一番楽ですが、あえてやっていません。DB のスキーマと API のスキーマを切り離すという判断です。

▶ (4) メタ情報つきレスポンス

```
{ data, page, limit, total, totalPages }
```

配列を裸で返さず、`data` でラップしてメタ情報を添えています。クライアントは `totalPages` を見てページャを描画できます。裸の配列で返すと、後からメタ情報を足したくなったときに**破壊的変更**になります。

`totalPages: Math.ceil(total / limit)` は端数の切り上げです。25件を20件ずつなら2ページになります。

5-3. attendanceService

```
export async function getMonthlyAttendances({ year, month }) {
  const attendances = await findAttendancesByMonth(year, month);
  return attendances.map((a) => ({
    date: toStr(a.date), // Date → "YYYY-MM-DD" 文字列に変換
    childId: a.childId, timeFrame: a.timeFrame,
    pickup: a.pickup, dropoff: a.dropoff,
  }));
}
```

ここでの重要な発想は `Date` を文字列に変換して返すことです。

JSON には `Date` 型が存在しないので、そのまま渡すと `"2026-08-06T00:00:00.000Z"` という ISO 文字列になり、クライアントで再パースが必要になります。`"2026-08-06"` にしておけば、クライアントは**文字列比較**だけで日付を照合でき、タイムゾーンの罠を

踏みません。

カレンダー側では `Map` のキーとして `"2026-08-06_3"`（日付_childId）のような形で引ける設計になります。

06 日付ロジック

ファイル: `src/lib/date.ts`。このアプリで最も複雑で、最も説明しづらい部分です。ここを説明できれば全体の理解が伝わります。

6-1. toStr — ローカル日付を YYYY-MM-DD に

```
export function toStr(date: Date): string {
  const year = date.getFullYear();
  const month = String(date.getMonth() + 1).padStart(2, "0");
  const day = String(date.getDate()).padStart(2, "0");
  return `${year}-${month}-${day}`;
}
```

要素	意味
<code>getMonth() + 1</code>	JS の <code>getMonth()</code> は0始まり (0=1月、11=12月)。人間向けには +1 が必要。JS 最大級の罠
<code>padStart(2, "0")</code>	<code>8</code> → <code>"08"</code> 。ゼロ埋めしないと文字列ソートが壊れる (<code>"2026-10-01"</code> < <code>"2026-8-01"</code> になる)
<code>getFullYear</code>	<code>getYear()</code> (非推奨、1900年起点) ではない正しい方

なぜ `toISOString()` を使わないのか

`toISOString()` は UTC に変換します。日本時間 (UTC+9) で `2026-08-06 00:00 JST` を `toISOString()` すると `"2026-08-05T15:00:00Z"` になり、前日の日付になってしまいます。

鉄則: 画面に表示する日付には、必ずローカル系の `getFullYear` / `getMonth` / `getDate` を使う。

6-2. getWeeks — カレンダーの心臓部

```
export function getWeeks(year: number, month: number): (Date | null)[][] {
  const weeks = new Map<string, (Date | null)[]>();
  const date = new Date(year, month, 1);

  while (date.getMonth() === month) {
    const dayOfWeek = date.getDay();
    if (dayOfWeek >= 1 && dayOfWeek <= 5) {
      const monday = new Date(date);
      monday.setDate(date.getDate() - (dayOfWeek - 1));
      const weekKey = toStr(monday);

      let week = weeks.get(weekKey);
      if (!week) {
```



```

        week = [null, null, null, null, null];
        weeks.set(weekKey, week);
    }
    week[dayOfWeek - 1] = new Date(date);
}
date.setDate(date.getDate() + 1);
}
return [...weeks.values()];
}

```

▶ 何をする関数か

「2026年8月」を渡すと、平日（月～金）だけを週ごとにまとめた2次元配列を返します。

```

[
  [null, null, 8/5, 8/6, 8/7 ], ← 8/1 が土曜始まりなので月・火は前月 → null
  [8/10, 8/11, 8/12, 8/13, 8/14],
  [8/17, 8/18, 8/19, 8/20, 8/21],
  ...
]

```

▶ ① グルーピングキーの正規化 — 最重要のアイデア

「同じ週かどうか」をどう判定するか。答えは「その日が属する週の月曜日を計算し、それをキーにする」です。

```
monday = date - (dayOfWeek - 1) 日
```

`dayOfWeek` は 月=1, 火=2 ... 金=5。水曜（3）なら 3-1=2 日戻して月曜、木曜（4）なら 3 日戻す。同じ週の日は全員、同じ月曜日に着地します。だから `toStr(monday)` が完璧なグルーピングキーになります。

これは汎用テクニックです

SQL の `GROUP BY` や `Lodash` の `groupBy` と同じ発想で、「分類したいものから、分類のキーを計算で導出する」というパターンです。週だけでなく「四半期」「時間帯」など、あらゆるグルーピングに応用できます。

▶ ② Map の挿入順序保証

`Map` はキーが挿入された順序を保持します（プレーンオブジェクトは整数風キーが先頭に並び替わるため使えません）。日付を1日ずつ昇順に走査しているので、週も自動的に時系列順に並びます。だから最後に `[...weeks.values()]` するだけでソート処理が不要です。

▶ ③ 固定長5の配列で「穴」を表現する

```

week = [null, null, null, null, null];
week[dayOfWeek - 1] = new Date(date);

```

新しい週を作るとき、先に `null` で5マス埋めておきます。そして曜日をそのままインデックスに使います（月=index 0、金=index 4）。

これにより、月初が水曜なら index 0, 1 が `null` のまま残り、カレンダーの空セルが自然に表現されます。

push で詰めると壊れる

`week.push(date)` で詰めていく実装だと、月初が水曜のとき水曜が index 0 に来てしまい、表示が2マス左にずれます。「曜日 → 配列インデックス」という位置の対応を維持することがこのロジックのキモです。

④ Date のミューテーションによるループ

```
const date = new Date(year, month, 1);
while (date.getMonth() === month) {
  ...
  date.setDate(date.getDate() + 1);
}
```

`setDate()` は桁あふれを自動処理します。8月31日に `setDate(32)` すると、勝手に9月1日になります。だから「その月が何日あるか」を計算する必要がなく、うるう年も自動対応します。終了条件が `getMonth() === month`なのは、この性質を利用しているためです。

参照 vs 値 — 典型的なバグの回避

`date` は同じオブジェクトを書き換え続けているので、配列に入れるときは必ず `new Date(date)` でコピーします。ここで `date` をそのまま入れると、全要素が最終日を指す同一オブジェクトになります。`monday` の計算でも `new Date(date)` してから `setDate` しているのは、元の `date` を壊さないためです。

⑤ 土日捨てる

`if (dayOfWeek >= 1 && dayOfWeek <= 5)` で土日をスキップしています。施設が平日営業だからで、ドメインの制約をロジックに反映している箇所です。

6-3. dateStrToDate / dateToDateStr — DB 用の変換ペア

```
export function dateStrToDate(dateStr: string): Date {
  return new Date(`${dateStr}T00:00:00Z`); // ← Z = UTC
}
export function dateToDateStr(date: Date): string {
  return date.toISOString().slice(0, 10); // ← UTC ベース
}
```

`toStr` がローカル時間ベース（表示用）なのに対し、こちらは UTC ベース（DB 用）です。この2系統を意図的に分けているのが設計のポイントです。

関数	基準	用途
<code>toStr</code>	ローカル	画面に表示する日付を作る
<code>dateStrToDate</code>	UTC	文字列を DB 保存用の Date にする
<code>dateToDateStr</code>	UTC	DB から出た Date を文字列にする

MySQL の `DATE` カラムは Prisma 経由で「UTC の 0 時」の `Date` オブジェクトとして往復します。だから DB に入れる／出す変換は UTC で統一する必要があります。

"2026-08-03" → `Date(2026-08-03T00:00:00Z)` → "2026-08-03" と往復して壊れないことを `date.test.ts` で検証しています。これを ラウンドトリップテスト と呼び、変換ペアの正しさを確かめる定番の手法です。

6-4. 現状の不整合（把握しておくべき点）

この2系統の使い分けが2箇所で崩れています

(a) `attendanceService.ts` で `toStr` を使っている

```
date: toStr(a.date) // ← DB から出た UTC 0 時 の Date に、ローカル系の関数を当てている
```

DB からは `2026-08-06T00:00:00Z` が返ります。日本（UTC+9）ではこれはローカル時間の 8/6 09:00 なので `toStr` は "2026-08-06" を返し、たまたま正しく動きます。しかし UTC より西のタイムゾーン（例：アメリカ）で動かすと 8/5 19:00 になり "2026-08-05" と1日ずれます。本来は `dateToDateStr` を使うべき箇所です。

(b) `attendanceRepository.ts` の範囲指定

```
const start = new Date(year, month, 1); // ← ローカル 0 時
const end = new Date(year, month + 1, 1);
```

DB のカラムは UTC 0 時なのに、比較する境界がローカル 0 時です。JST では境界が9時間前にずれるため結果的に月の範囲が合いますが、これもタイムゾーン依存です。`dateStrToDate("2026-08-01")` のような UTC 基準の生成に揃えるのが安全です。

「なぜ今は動いているのか」まで説明できると、理解の深さが伝わります。

07 バリデーション層（Zod）

7-1. スキーマ定義

ファイル： `src/schemas/childSchema.ts`

```
export const createChildSchema = z.object({
  name: z.string().min(1, "名前を入力してください").max(20, "名前は20文字までです"),
  color: z.string().regex(/^#[0-9a-fA-F]{6}$/, "色は #RRGGBB 形式で指定してください"),
  contractDays: z.number().int().min(0).max(13),
  sortOrder: z.number().int().min(0),
});

export type CreateChildInput = z.infer<typeof createChildSchema>;
```

▶ 中核の発想：スキーマファースト / Single Source of Truth

`z.infer<typeof createChildSchema>` がこの章で最も重要です。スキーマから TypeScript の型を導出しています。

```
type CreateChildInput = {
  name: string; color: string; contractDays: number; sortOrder: number;
}
```

なぜこれが強力か

普通なら「型定義」と「バリデーション」を別々に書き、**片方だけ直してズレる**という事故が起きます。Zod ではスキーマが唯一の真実の源であり、型は自動で追従します。**1箇所直せば全部直る**。

さらに、この 1 つのスキーマが**3箇所**で使われています。

使用箇所	用途	目的
<code>ChildForm.tsx</code> の <code>zodResolver</code>	クライアント側検証	UX（すぐエラーが出て親切）
<code>api/children/route.ts</code> の <code>safeParse</code>	サーバー側検証	セキュリティ（不正入力の遮断）
<code>z.infer</code>	TypeScript の型	開発時の型安全

両方で検証する理由

クライアント検証は `curl` や `DevTools` で**必ず迂回**できます。だからサーバー検証は絶対に省略できません。逆にサーバー検証だけだと、入力ミスのたびに通信が発生してUXが悪くなります。**役割が違うので両方必要**です。

▶ 正規表現による色の検証

`/^#[0-9a-fA-F]{6}$/` で `#4a86c4` 形式のみを許可します。`^` と `$` で**全体一致**を強制しているのが重要で、これがないと `"xxx#4a86c4yyy"` が通過してしまいます。

▶ エラーメッセージを日本語で埋め込む

`min(1, "名前を入力してください")` のように、バリデーションルールとメッセージを同じ場所に置いています。これでフォームと API で同じ文言が出せ、文言の管理場所も 1 箇所になります。

7-2. クエリパラメータのスキーマ

```
export const childListQuerySchema = z.object({
  page: z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20),
});
```

▶ `z.coerce.number()` — 型強制

URL のクエリパラメータは**必ず文字列**です（`?page=2` → `"2"`）。`z.coerce` が `Number("2")` を内部で実行してから検証します。これがないと常に型エラーになります。

▶ `.default(1)` — デフォルト値

`?page=` を省略しても `1` が入ります。パースとデフォルト適用を同じ場所でやるので、後続のコードは「値が必ず存在する」前提で書けます。`page ?? 1` をあちこちに書かずに済みます。

▶ `.max(100)` — 地味だが重要な防御

`?limit=999999` を弾きます。これがないと、悪意ある（あるいはうっかりの）リクエストで DB から全件取得されてサーバーが落ちます。リソース枯渇攻撃への防御です。

7-3. `safeParse` と `parse` の違い

メソッド	失敗時の挙動	適した場面
<code>parse()</code>	例外を投げる	「起こり得ないはず」の内部データ
<code>safeParse()</code>	<code>{ success, data error }</code> を返す	外部入力（採用）

```
const parsed = createChildSchema.safeParse(body);
if (!parsed.success) {
  return Response.json({ errors: toFieldErrors(parsed.error) }, { status: 400 });
}
// ここから下では parsed.data が型安全にアクセスできる
```

`safeParse` を使うと、TypeScript の判別可能ユニオン（discriminated union）により、`if (!parsed.success)` を抜けた後は `parsed.data` が型安全にアクセスできます。API では「400 を返す」というのは正常な分岐なので、例外ではなく戻り値で扱うのが適切です。

7-4. エラー変換 — 腐敗防止層

ファイル： `src/lib/apiError.ts`

```
export function toFieldErrors(error: ZodError): FieldError[] {
  return error.issues.map((issue) => ({
    field: issue.path.join("."),
    message: issue.message,
  }));
}
```

Zod の内部エラー構造を、API の公開契約である `FieldError[]` に変換する腐敗防止層（Anti-Corruption Layer）です。

- `issue.path` は `["address", "zip"]` のような配列 → `join(".")` で `"address.zip"` にする。ネストしたオブジェクトにも対応したドット記法
- Zod のバージョンが変わって内部構造が変化しても、この関数だけ直せば API の形は保たれる
- クライアントは `errors[0].message` で表示、`errors[0].field` でどの入力欄かを特定できる

`{ errors: [...] }` という一貫したエラー形式を全 API で使っているのも設計判断です。クライアント側のエラー処理を1つに統一できます。

08 認証とアクセス制御

このアプリは3つの層で認可しています。これは**多層防御（Defense in Depth）**の実践です。1つが破られても次で止まります。

8-1. 層1：proxy.ts — 全ページの入口

ファイル：`src/proxy.ts`

```
export function proxy(request: NextRequest) {
  const hasSession =
    request.cookies.has("authjs.session-token") ||
    request.cookies.has("__Secure-authjs.session-token");

  if (!hasSession) return NextResponse.redirect(new URL("/login", request.url));
  return NextResponse.next();
}

export const config = {
  matcher: ["/((?!api|_next|favicon.ico|login).*)"],
};
```

proxy.ts という名前 — Next.js 16 の破壊的変更

従来 `middleware.ts` / `export function middleware` だったものが、Next.js 16 で `proxy.ts` / `export function proxy` にリネームされました。役割は同じで、**全リクエストがルーティングされる前に通る関門**です。

matcher の正規表現

```
/((?!api|_next|favicon.ico|login).*)
```

`(?!...)` は**否定先読み（negative lookahead）**。「`api`・`_next`・`favicon.ico`・`login` で始まらない全パス」にマッチします。

除外対象	理由
<code>api</code>	API は自前で <code>requireApprovedUser()</code> を呼ぶ。また API はリダイレクトではなく 401 JSON を返すべき
<code>_next</code>	JS / CSS などのビルド成果物。認証をかけるとページが描画できなくなる
<code>login</code>	ここもブロックしたら 無限リダイレクトループ になる
<code>favicon.ico</code>	静的ファイル。認証不要

▶ Cookie の存在だけを見て、中身を検証していない

これは意図的な設計です

proxy は全リクエストで走るので、ここで DB を叩くと**全ページが遅くなります**。だから「Cookie すら無い明らかな未ログインを弾く」という粗いフィルタに徹し、本当の検証は下の層に任せています。**性能と安全性のトレードオフ**を理解した設計です。Cookie を偽造すれば proxy は通過できますが、次の層で必ず落ちます。

▶ 2種類の Cookie 名

`authjs.session-token` (HTTP・開発環境) と `__Secure-authjs.session-token` (HTTPS・本番)。`__Secure-` はブラウザが強制する **Cookie 接頭辞**で、HTTPS 経由でしかセットできません。両方チェックするのは開発と本番の両対応のためです。

8-2. 層2：Server Component

```
const session = await auth();
if (!session) redirect("/login");
```

`page.tsx` で、DB のセッションテーブルを実際に照会します。Cookie の中身が本物かをここで初めて確認します。

8-3. 層3：requireApprovedUser — API のガード

ファイル： `src/lib/requireApprovedUser.ts`

```
export async function requireApprovedUser() {
  const session = await auth();
  if (!session?.user?.id) {
    return { ok: false, status: 401, message: "ログインが必要です" } as const;
  }
  const user = await findUserById(session.user.id);
  if (!user) {
    return { ok: false, status: 401, message: "ログインが必要です" } as const;
  }
  if (!user.approved) {
    return { ok: false, status: 403, message: "利用が承認されていません" } as const;
  }
  return { ok: true, user } as const;
}
```

▶ 判別可能ユニオン + as const

`as const` により、戻り値の型が**リテラル型のユニオン**になります。

```
| { ok: false; status: 401; message: "ログインが必要です" }
| { ok: false; status: 403; message: "利用が承認されていません" }
| { ok: true; user: User }
```


`ok` が判別子（discriminant）なので、`if (!authResult.ok)` を抜けた後は `authResult.user` が型安全に使えます。TypeScript が自動で型を絞り込みます（narrowing）。`as const` がないと `ok: boolean` になり、絞り込みが効きません。

▶ 401 と 403 の使い分け — HTTP の正しい理解

コード	意味	解決方法
401 Unauthorized	あなたが誰か分からない	ログインすれば解決する
403 Forbidden	誰かは分かるが権限がない	ログインし直しても無駄。管理者の承認が必要

`approved: false` は「本人確認はできているが許可されていない」状態なので **403** です。ここを正確に使い分けているのは good practice です。（401 の名前が Unauthorizedなのは歴史的な誤りで、実際の意味は Unauthenticatedです。）

▶ セッションを信じず、DB を再確認する

```
const user = await findUserById(session.user.id);
```

セッションが有効でも、**その都度 DB の `approved` を読み直します**。管理者が承認を取り消した瞬間から使えなくなります。もし `approved` をセッションにキャッシュしていたら、セッション期限まで使い続けられてしまいます。

▶ 例外ではなく戻り値でエラーを表現する

`throw new UnauthorizedError()` ではなく `{ ok: false, ... }` を返す設計です。呼び出し側が `try / catch` を書かずに済み、型システムが「エラー処理を書き忘れているか」をチェックできます（`authResult.user` は `ok` チェックを通らないとアクセスできない）。Rust の `Result` 型に近い発想です。

8-4. 認証本体

ファイル： `src/auth.ts`

```
export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(prisma),
  providers: [GitHub],
  session: { strategy: "database" },
});
```

- **分割代入エクスポート** — NextAuth v5 は設定を1箇所に書いて4つの道具を取り出す設計。v4 の「`[...nextauth].ts` に全部書く」方式から改善されました
- **Adapter パターン** — `PrismaAdapter` が「Auth.js が要求する保存操作」と「Prisma」の橋渡しをします。Drizzle や MongoDB に差し替え可能です
- **OAuth 2.0 / OpenID Connect** — GitHub にパスワードを預け、自分は持ちません。**パスワードを保存しないことが最強のパスワード管理**という発想です。漏洩しようがありません

8-5. キャッチオールルート

ファイル: `src/app/api/auth/[...nextauth]/route.ts`

```
export const { GET, POST } = handlers;
```

`[...nextauth]` はキャッチオールセグメントです。`/api/auth/signin`、`/api/auth/callback/github`、`/api/auth/session` など、Auth.js が使う複数のエンドポイントを1ファイルで受けます。中身は Auth.js に丸投げで、実質2行です。

09 API ルート（Route Handler）

ファイル：`src/app/api/children/route.ts`。処理の順番そのものが設計になっています。

```
export async function POST(request: Request) {
  const logger = createLogger({ requestId: crypto.randomUUID() }); // ①
  const authResult = await requireApprovedUser(); // ②
  if (!authResult.ok) {
    return Response.json({ error: authResult.message }, { status: authResult.status });
  }

  logger("info", "child.create.start"); // ③
  const body = await request.json();
  const parsed = createChildSchema.safeParse(body); // ④

  if (!parsed.success) {
    const errors = toFieldErrors(parsed.error);
    logger("warn", "child.create.failed", { fields: errors.map((e) => e.field) });
    return Response.json({ errors }, { status: 400 });
  }

  const child = await addChild(parsed.data); // ⑤
  logger("info", "child.create.success", { childId: child.id });
  return Response.json(child, { status: 201 }); // ⑥
}
```

9-1. 認証 → 検証 → 実行 という順番

なぜ認証が先か

未認証ユーザーにバリデーションエラーの詳細を教えないためです（情報漏洩の最小化）。「どのフィールドが必須か」も攻撃者にとっては情報になります。また、無駄な処理を早期に打ち切れます。

この「ガード節を積み上げて、正常系を最後に置く」書き方は、ネストが深くならず読みやすくなります。`if (ok) { if (valid) { ... } }` と入れ子にする書き方の対極です。

9-2. Web 標準 API を使っている

`Request` / `Response` は Next.js 独自ではなく **Web 標準（Fetch API）** です。`request.json()`、`new URL(request.url)`、`Response.json()` はブラウザにもあるのと同じ API です。

Express の `req` / `res` と違い、Deno や Cloudflare Workers でも通用する知識になります。`Response.json(data, { status: 201 })` は `Content-Type: application/json` を自動で付けてくれる静的メソッドです。

9-3. Object.fromEntries(searchParams)

```
const { searchParams } = new URL(request.url);
const parsed = childListQuerySchema.safeParse(Object.fromEntries(searchParams));
```

`URLSearchParams` はイテラブルで `[["page", "2"], ["limit", "20"]]` を吐きます。`Object.fromEntries` でプレーンオブジェクトに変換し、Zod に渡します。1行でクエリ全体を扱える定番イディオムです。
(同じキーが複数ある場合は最後の値だけ残りますが、このアプリでは問題になりません。)

9-4. HTTP ステータスコードの使い分け

コード	意味	使用箇所
200	OK	GET 成功
201	Created	POST 成功時 (リソースを作成した)
400	Bad Request	Zod 検証失敗
401	Unauthorized	セッションなし
403	Forbidden	<code>approved: false</code>

POST で 200 ではなく **201** を返しているのが丁寧なポイントです。「作成された」ことがステータスコードだけで伝わります。

9-5. 構造化ログ

ファイル: `src/lib/logger.ts`

```
export function log(level, message, context = {}) {
  console.log(JSON.stringify({
    time: new Date().toISOString(), level, message, ...context
  }));
}

export function createLogger(base: Record<string, unknown>) {
  return (level, message, context = {}) =>
    log(level, message, { ...base, ...context });
}
```

▶ JSON 1行ログ (構造化ログ)

`console.log("児童を作成しました: " + id)` のような文字列ログではなく、**機械可読な JSON** を1行で出します。CloudWatch / Datadog / Loki などのログ基盤がそのままフィールドとして検索・集計できるようになります。「`level: "warn"` かつ `message: "child.create.failed"` を日別に集計」といったクエリが書けます。

▶ 高階関数によるコンテキスト注入

`createLogger` は関数を返す関数（高階関数）です。クロージャで `base` を捕まえておき、以降の全ログに自動で混ぜ込みます。

```
const logger = createLogger({ requestId: crypto.randomUUID() });
```

相関ID（Correlation ID）

これでそのリクエスト中の全ログに同じ `requestId` が入ります。大量のログの中から「あの1リクエストで何が起きたか」を追跡できるようになります。分散システムのデバッグでは必須の技術です。

`{ ...base, ...context }` の順序も意図的で、後から渡した `context` が `base` を上書きできるようになっています。

▶ イベント名の命名規則

`child.create.start` / `child.create.failed` / `child.create.success` というドット区切りの階層的な名前を使っています。`child.create.*` で前方一致検索でき、`.failed` で横断検索もできます。日本語の自由文だと検索も集計もできません。

▶ ログの内容への配慮

```
logger("warn", "child.create.failed", { fields: errors.map((e) => e.field) });
```

フィールド名だけをログに出し、`name` の実際の値（＝児童の氏名という個人情報）は出していません。個人情報をログに残さないという配慮です。ログは長期保存され、アクセス権も広くなりがちなので重要です。

▶ `crypto.randomUUID()`

Web 標準 API です。ライブラリ（uuid パッケージ）不要で、暗号学的に安全な UUID v4 を生成します。

10 クライアント層

10-1. Server Component と Client Component の境界

App Router で最も重要な概念です。

	Server Component (既定)	Client Component ("use client")
実行場所	サーバーのみ	サーバー (初回HTML) + ブラウザ
DB 直アクセス	できる	できない
<code>useState</code> / <code>useEffect</code>	使えない	使える
onClick 等のイベント	使えない	使える
JS バンドル	含まれない	含まれる

▶ このアプリの割り当て

ファイル	種別	理由
<code>layout.tsx</code>	Server	静的な骨組みのみ
<code>page.tsx</code>	Server async	<code>auth()</code> で DB を直接読む
<code>Calendar.tsx</code>	Server	計算と描画のみ、状態を持たない
<code>Providers.tsx</code>	Client	<code>useState</code> と Context が必要
<code>ChildList.tsx</code>	Client	<code>useQuery</code> フックを使う
<code>ChildForm.tsx</code>	Client	フォームの状態管理

原則: "use client" はできるだけ葉 (末端) に置く

"use client" を付けたコンポーネントの **import 先も全部 Client になります**。上のほうに付けるとアプリ全体がクライアントに落ち、JS バンドルが太ります。

▶ 重要: children による境界の突破

`layout.tsx` では `Providers` (Client) が全体を包んでいます。

```
<body><Providers>{children}</Providers></body>
```

ここで疑問が湧くはず — 「じゃあ `page.tsx` もクライアントになるのでは？」

なりません

`children` として `props` 経由で渡された Server Component は Server のままです。Providers はそれを「すでにレンダリング済みの結果」として受け取って表示だけです。

これが `children` パターン（コンポジション）で、Client Provider を使いつつサーバーレンダリングを維持する App Router の最重要テクニックです。「import した子は Client になるが、`children` で渡された子はない」と覚えます。

10-2. layout.tsx

```
const geistSans = Geist({ variable: "--font-geist-sans", subsets: ["latin"] });

export const metadata: Metadata = {
  title: "ひまわり予定表",
  description: "療育施設の日程を管理するWebアプリ",
};
```

▶ next/font によるフォント最適化

ビルド時にフォントをダウンロードして自己ホスティングします。効果は3つです。

- **プライバシー** — Google のサーバーにユーザーの IP が渡らない
- **速度** — 外部への DNS 解決・TLS 接続が不要になる
- **CLS 防止** — レイアウトシフトを防ぐためのフォールバックフォント調整を自動で行う

▶ その他の要素

- `variable: "--font-geist-sans"` — CSS 変数として出力し、`<html className={geistSans.variable}>` で流し込む。Tailwind の `@theme` から参照できる
- `subsets: ["latin"]` — 使う文字だけダウンロードしてファイルサイズを削減
- `metadata` エクスポート — Next.js が `<head>` に `<title>` と `<meta>` を自動挿入する宣言的 API
- `lang="ja"` — アクセシビリティ（スクリーンリーダーの読み上げ言語）に必須
- `Readonly<{ children: React.ReactNode }>` — `React.ReactNode` は「React が描画できるもの全部」（JSX、文字列、数値、null、配列）を表す最も広い型
- `h-full` + `flex min-h-full flex-col` — フッターを最下部に固定する定番の flex レイアウト

10-3. page.tsx

```
export default async function Home({
  searchParams,
}): {
  searchParams: Promise<{ year?: string; month?: string }>;
} {
```

```

const session = await auth();
if (!session) redirect("/login");

const params = await searchParams;
const now = new Date();
const year = Number(params.year) || now.getFullYear();
const month = params.month ? Number(params.month) - 1 : now.getMonth();
...
}

```

▶ async なコンポーネント

React 19 + Server Components の機能です。コンポーネント関数自体が `async` で、中で `await` できます。従来は `useEffect` + `useState` でやっていたデータ取得が、`await` 1行になります。

▶ searchParams が Promise である理由

Next.js 15 の破壊的変更です。以前は同期オブジェクトでしたが、部分プリレンダリング対応のため非同期化されました。`await searchParams` が必要です。

▶ URL をアプリの状態にする（URL as State）

表示中の年月を `useState` ではなく URL のクエリパラメータで持っています。

```
/?year=2026&month=9
```

得られるもの	内容
共有・ブックマーク	URL を送れば相手も同じ画面が開く
ブラウザの戻る／進む	そのまま効く
リロード耐性	更新しても表示月が消えない
コード量	状態管理コードが一切不要

`Calendar.tsx` の前月／翌月が `<button onClick>` ではなく `<Link href>`なのはこのためです。JS が無効でも動き、右クリックで新規タブも開けます。

▶ フォールバック値の書き分け — || の罫

```

const year = Number(params.year) || now.getFullYear();
const month = params.month ? Number(params.month) - 1 : now.getMonth();

```


なぜ month だけ書き方が違うのか

year は || で問題ありません（年に 0 はあり得ないため）。

しかし month は || が使えません。`?month=1`（1月）→ `1 - 1 = 0` となり、`0 || now.getMonth()` は 0 が falsy なので今月に化けます。だから三項演算子で「パラメータが存在するか」を先に判定しています。

- 1 は URL（人間向け 1～12）と JS の月（0～11）の変換です。境界で変換して、内部は 0 始まりで統一する方針になっています。

▶ Server Action

```
<form action={async () => { "use server"; await signOut(); }}>
  <button type="submit">ログアウト</button>
</form>
```

Server Action（React 19）です。関数の中に `"use server"` を書くと、その関数はサーバーでのみ実行され、クライアントには「呼び出すための ID」だけが渡ります。React が内部で POST リクエストを生成します。

- `<form action={...}>` に渡しているので、JS が読み込まれる前でも動作します（プログレッシブエンハンスメント）
- `onClick` + `fetch` と違い、API ルートを作る必要がありません
- ログアウトを `<a href>` ではなく `<form>` + POST にしているのは正しく、GET は副作用を持つてはいけない（ブラウザやクローラーが勝手に叩く可能性がある）という HTTP の原則に沿っています

10-4. Providers.tsx

```
"use client";
export function Providers({ children }: { children: React.ReactNode }) {
  const [queryClient] = useState(() => new QueryClient());
  return (
    <QueryClientProvider client={queryClient}>
      {children}
      <Toaster richColors position="top-center" />
    </QueryClientProvider>
  );
}
```

▶ useState() => new QueryClient() — 2つの罠の回避

罠1：モジュールスコープで作ってはいけない

```
const queryClient = new QueryClient(); // × 危険
```

サーバーでは同じモジュールが全ユーザーで共有されます。A さんのデータがキャッシュされ、B さんに漏れます。深刻なセキュリティ問題です。useState に入れることでコンポーネントインスタンスごとに1つになります。

⚠ 2: `useState(new QueryClient())` ではない

```
const [queryClient] = useState(new QueryClient()); // × 無駄
```

これだと再レンダリングのたびに `new QueryClient()` が実行されます（結果は捨てられますが、生成コストは払います）。`useState(() => ...)` の関数を渡す形（遅延初期化 / lazy initializer）なら、React は初回だけ関数を呼びます。

▶ Provider パターン（React Context）

`QueryClientProvider` は Context を使って、配下のどの深さのコンポーネントからも `useQueryClient()` で client を取れるようにします。props を何段も手渡しする **prop drilling** を回避する仕組みです。

▶ Toaster をここに1つだけ置く

トースト通知の表示場所を1箇所に置き、各コンポーネントからは `toast.success(...)` を呼ぶだけにしています。「通知を出したい人」と「通知を表示する人」を分離する設計です。`position="top-center"` で表示位置、`richColors` で成功 = 緑・失敗 = 赤の色分けが有効になります。

10-5. useChildList — データ取得フック

```
async function fetchChildList(): Promise<ChildListResponse> {
  const res = await fetch("/api/children?limit=100");
  if (!res.ok) throw new Error("子供の取得に失敗しました");
  return res.json();
}

export function useChildList() {
  return useQuery({ queryKey: ["children"], queryFn: fetchChildList });
}
```

▶ if (!res.ok) throw — fetch の有名な落とし穴

fetch は 404 や 500 でも reject しません

reject するのはネットワークが切断されたときだけです。`res.ok`（ステータスが 200～299 か）を自分でチェックして例外にする必要があります。React Query はこの例外を捕まえて `isError` にします。

▶ queryKey: ["children"] — キャッシュの住所

React Query の中核概念です。このキーがキャッシュのキーであり、

- 同じキーの `useQuery` はキャッシュを共有する（複数コンポーネントで呼んでも fetch は1回）
- `invalidateQueries({ queryKey: ["children"] })` でこのキーのキャッシュを無効化できる

配列なのは階層を表すためです。`["children", { page: 2 }]` のようにパラメータを含めると、パラメータごとに別キャッシュになり、キーが変わると自動で再取得されます。

▶ カスタムフックに切り出す発想

`useQuery(...)` をコンポーネントに直接書かず `useChildList()` に包む理由：

- `queryKey` のタイプミスを防ぐ（1箇所に集約）
- `fetch` の URL やエラーメッセージも1箇所にまとまる
- 複数のコンポーネントから使い回せる
- 後で「React Query をやめる」となってもフックの中だけ直せばよい（ロジックのカプセル化）

これが **Custom Hook パターン**で、React における「ロジックの再利用単位」です。

▶ React Query が自動でやってくれること

自分で `useEffect` + `useState` で書くと自前実装が必要になるものが、全部入っています。

- ローディング状態 / エラー状態の管理
- キャッシュ — 2回目以降は即座に表示
- 重複排除（deduplication） — 同時に同じクエリが走らない
- ウィンドウ復帰時の再取得 — タブを戻ると最新化
- リトライ — 失敗時に自動再試行
- 競合状態の回避 — 古いレスポンスが新しいのを上書きする問題を防ぐ

10-6. useAddChild — 更新フック

```
async function postChild(input: CreateChildInput): Promise<Child> {
  const res = await fetch("/api/children", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(input),
  });
  if (!res.ok) {
    const body = await res.json().catch(() => null);
    const message =
      body?.errors?.[0]?.message ?? body?.error ?? "登録に失敗しました";
    throw new Error(message);
  }
  return res.json();
}

export function useAddChild() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: postChild,
```

```

onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ["children"] });
  toast.success("登録しました");
},
onError: (error) => toast.error(error.message),
});
}

```

▶ useQuery と useMutation の使い分け

フック	対象	実行タイミング
<code>useQuery</code>	読み取り (GET)	自動で実行され、キャッシュされる
<code>useMutation</code>	書き込み (POST/PUT/DELETE)	<code>mutate()</code> を呼んだときだけ実行、キャッシュしない

「副作用のある操作は勝手に実行してはいけない」という区別です。

▶ 三段のエラーメッセージフォールバック

```
body?.errors?.[0]?.message ?? body?.error ?? "登録に失敗しました"
```

段	形式	発生源
1	<code>{ errors: [{ field, message }] }</code>	Zod のフィールドエラー (400)
2	<code>{ error: "..." }</code>	認証エラー (401 / 403)
3	固定文言	それ以外 (500 で HTML が返る等)

オプションルチェン `?.` で途中が `undefined` でも落ちず、`??` で順に次を試します。3種類のエラー形式を1行で吸収しています。

`await res.json().catch(() => null)` も重要で、サーバーが JSON を返さなかった場合 (500 で HTML エラーページが返る等) に `res.json()` が例外を投げるのを防いでいます。エラー処理の中でさらにエラーが出るのを防ぐ配慮です。

▶ キャッシュ無効化 (Invalidation) — フォームと一覧をつなぐ唯一の線

```
onSuccess: () => queryClient.invalidateQueries({ queryKey: ["children"] })
```

ここが React Query を使う最大の理由

追加成功 → `["children"]` のキャッシュを「古い」とマーク → 画面に表示中の `useChildList` が自動で再取得 → 一覧に新しい児童が現れる。

`ChildForm` と `ChildList` は互いを一切知りません。props も渡していません。queryKey という共通言語だけで疎結合に連携しています。

発展形として、`setQueryData` で手でキャッシュに追加する方法や、**楽観的更新** (optimistic update) でサーバー応答を待たず即座に画面反映する方法もあります。

▶ onSuccess が2箇所にある理由

```
// フック側
onSuccess: () => { invalidateQueries(); toast.success(...); }

// ChildForm 側
addChild.mutate(values, { onSuccess: () => reset() });
```

両方とも実行されます。役割分担が明確です。

- フック側 = どこから呼んでも必要な共通処理 (キャッシュ更新、通知)
- 呼び出し側 = その画面固有の処理 (フォームのリセット)

「共通の関心事はフックに、固有の関心事は呼び出し側に」という**関心の分離**です。

10-7. ChildForm.tsx

```
const { register, handleSubmit, reset, formState: { errors } } =
  useForm<CreateChildInput>({
    resolver: zodResolver(createChildSchema),
    defaultValues: {
      name: "", color: "#4a86c4", contractDays: 0, sortOrder: 0,
    },
  });
```

▶ 非制御コンポーネント — React Hook Form の設計思想

```
<input {...register("name")} />
```

`register("name")` は `{ name, onChange, onBlur, ref }` を返し、スプレッドで input に展開されます。ポイントは `ref` で DOM 要素を直接参照していることです。

	制御コンポーネント (useState)	非制御 (RHF・採用)
値の保持場所	React の state	DOM 自身
1文字入力するたび	再レンダリング	再レンダリングなし
コード量	項目ごとに state + onChange	<code>{...register("name")}</code> のみ

フォームの項目が増えるほど差が出ます。「入力のたびにフォーム全体が再描画される」のを避けるパフォーマンス最適化が設計に組み込まれています。

▶ **handleSubmit(onSubmit) — 高階関数**

`handleSubmit` が次の5つをやってくれます。

- 1. `event.preventDefault()` (ページリロードを防ぐ)
- 2. 全フィールドの値を集める
- 3. **Zod で検証する**
- 4. 成功したときだけ `onSubmit(values)` を呼ぶ
- 5. 失敗なら `errors` を埋める

だから `onSubmit` は**検証済みの正しいデータ**だけを受け取れます。型も `CreateChildInput` として保証されます。

▶ **zodResolver — アダプターパターン**

RHF は Zod を知りません。Yup、Joi、Valibot など何でも使えるようにリゾルバというインターフェースを切っています。`zodResolver` はその実装で、RHF と Zod の**接続アダプター**です。

これにより `schemas/childSchema.ts` の 1 つのスキーマが、クライアント検証にもサーバー検証にも使えます。**二重管理ゼロ**です。

▶ **valueAsNumber: true**

```
<input type="number" {...register("contractDays", { valueAsNumber: true })} />
```

HTML の input の値は `type="number"` でも常に文字列

"5" が返ってきます。Zod スキーマは `z.number()` を要求しているので、そのままだと**必ず検証エラー**になります。`valueAsNumber` が `Number()` 変換をかけます。

サーバー側 (クエリ) では `z.coerce.number()`、クライアント側では `valueAsNumber` と、それぞれの場所に合った型変換をしています。

▶ **ジェネリクス useForm<CreateChildInput>**

型パラメータを渡すことで、

- `register("nmae")` のタイプミスがコンパイルエラーになる
- `errors.name` が補完される
- `onSubmit(values)` の `values` に型が付く

Zod スキーマから導出した型が、フォームの隅々まで行き渡ります。

▶ その他の配慮

- `disabled={addChild.isPending}` — 送信中はボタンを無効化して二重送信を防ぐ。`isPending` は React Query が管理してくれる状態
- `reset()` — `defaultValues` で指定した初期値に戻す。連続で複数の児童を登録する運用を想定した配慮

10-8. ChildList.tsx と Skeleton

```
const { data, isPending, isError } = useChildList();

if (isPending) return <ChildListSkeleton />;
if (isError) return <p>エラーが発生しました</p>;

return (
  <ul>
    {data.data.map((child) => (
      <li key={child.id} style={{ color: child.color }}>
        {child.name} (契約 {child.contractDays} 日)
      </li>
    ))}
  </ul>
);
```

▶ 3状態の網羅（early return）

読み込み中 / エラー / 成功の3つを漏れなく扱っています。early return で先に2つを処理するので、

ガード節が型安全にも貢献する

最後の `return` の時点で TypeScript は `data` が `undefined` でないと判断できます（型の絞り込み）。だから `data.data.map` が `data?.data?.map` なしで書けます。

▶ `key={child.id}` — インデックスを使わない理由

React のリストレンダリングに必須です。配列のインデックスではなく DB の id を使うのが重要で、インデックスだと並び替えや途中削除が起きたときに React が要素を取り違え、入力中の値が別の行に移る等のバグになります。

▶ style={{ color: child.color }} — ここだけインラインスタイル

色は**実行時に DB から来る動的な値**なので、Tailwind のクラス（ビルド時に静的解析される）では表現できません。ここだけインラインスタイルを使うのは正しい判断です。

▶ Skeleton UI

```
{[0, 1, 2].map((i) => (  
  <li key={i} className="h-5 w-48 animate-pulse rounded bg-gray-200" />  
))}
```

スピナーではなく**中身の形を模した灰色のブロック**を出す手法です。

- 知覚性能（perceived performance）の向上 — 「何が来るか」が見えているので体感待ち時間が短い
- CLS（Cumulative Layout Shift）の抑制 — 先に高さ（`h-5`）を確保しておくので、データ到着時に画面がガタつかない。Google の Core Web Vitals の指標でもある
- `animate-pulse` — Tailwind 組み込みの点滅アニメーション。「読み込み中である」ことを伝える

ここでは中身が単なるプレースホルダーなので `key={i}`（インデックス）で問題ありません。

ファイル名のタイポ

`ChildListSkelton.tsx` — 正しくは `Skeleton` です。エクスポートされている関数名は `ChildListSkeleton`（正しい綴り）なので、**ファイル名だけがズレている状態**です。動作はしますが直したほうがよい点です。

10-9. Calendar.tsx

```
const Calendar = ({ year, month }: { year: number; month: number }) => {  
  const weeks = getWeeks(year, month);  
  const prev = new Date(year, month - 1, 1);  
  const next = new Date(year, month + 1, 1);
```

▶ props 設計 — プレゼンテーションコンポーネント

`year` と `month` の**数値2つだけ**を受け取ります。「予定データ」も「取得関数」も受け取りません。

- 純関数的（同じ props なら常に同じ出力）
- テストが極めて簡単
- Server Component のままでいられる（`"use client"` 不要 → JS バンドルに含まれない）

▶ new Date(year, month - 1, 1) — 桁あふれの利用

`month` が 0（1月）のとき `month - 1` は `-1` ですが、`new Date(2026, -1, 1)` は自動的に**2025年12月**になります。`if (month === 0) { year--; month = 11 }` のような分岐が不要です。11 → 12 も同様に翌年1月になります。**Date** コンストラクタの正規化を利用した簡潔な書き方です。

▶ テーブルによるカレンダー

`<table>` / `<thead>` / `<tbody>` を使っているのはセマンティックに正しい選択です。カレンダーは本質的に「曜日 × 週」の2次元表なので、div のグリッドより意味が明確で、スクリーンリーダーにも構造が伝わります。

クラス	効果
<code>table-fixed</code>	列幅を均等に固定（内容によって幅が変わらない）
<code>border-separate border-spacing-1.5</code>	セル間に隙間を作り、カード状の見た目にする
<code>{date ? date.getDate() : ""}</code>	<code>getWeeks</code> が返した <code>null</code> （月外の平日）を空セルとして描画

▶ btnClass を変数に切り出す

Tailwind のクラス文字列が長くなるので変数化しています。前月・翌月の2箇所です使うので DRY になります。本格的にやるなら `className` や `classNames` を使いますが、2箇所ならこれで十分です。

▶ Link によるクライアントサイドナビゲーション

`<a>` ではなく `next/link` の `<Link>` を使っています。ページ全体をリロードせずに遷移し、さらにビューポートに入った時点で遷移先をプリフェッチします。前月／翌月の切り替えが体感で即座になります。

11 型の流れ

このアプリで一番の技術的な芯は、**型がどこから来てどこへ行くか**です。源流が3つあり、それらを意図的に分離しています。

```
① prisma/schema.prisma
  ↓ prisma generate
src/generated/prisma/*    (Prisma.ChildCreateInput、Child モデル型...)
  ↓
Repository の引数・戻り値

② src/schemas/childSchema.ts (Zod)
  ↓ z.infer
CreateChildInput
  ↓
ChildForm の      postChild の      route.ts の
useForm<T>         引数の型         safeParse

③ src/types/api.ts (手書きの API 契約)
  ↓
Service の戻り値 ↔ useChildList の Promise<T>
```

11-1. ① と ③ を分けている理由

① が DB のスキーマ、③ が API のスキーマです。Service の `map` が ①→③ の変換点になっており、DB スキーマの変更が API に自動で漏れない構造になっています。

もし分けなかったら

`Child` テーブルに `medicalNote` (医療的ケアの記録) を追加した瞬間、API のレスポンスにも自動的に含まれて外部に出ます。分けていれば、明示的に `types/api.ts` と Service の `map` を変更しない限り漏れません。

11-2. ② だけが実行時にも存在する

TypeScript の型はビルド時に消えます。だから外部から来るデータ (HTTP ボディ、クエリパラメータ) は Zod で実行時に検証しなければ安全になりません。

型注釈は嘘をつけるが、Zod は嘘をつけない。

```
const body = await request.json() as CreateChildInput
```

と書いても、実際に何が入っているかは保証されません。`safeParse` だけが実際に中身を確認します。

11-3. types/api.ts — API 契約の明文化

```
export type FieldError = { field: string; message: string; };

export type Child = {
  id: number; name: string; color: string;
  contractDays: number; sortOrder: number;
};

export type ChildListResponse = {
  data: Child[]; page: number; limit: number;
  total: number; totalPages: number;
};
```

この型がサーバー（Service の戻り値）とクライアント（`useChildList` の `Promise<T>`）の両方で使われているのがポイントです。同じリポジトリ内なので import 1本で共有でき、API の形が変わればサーバーとクライアントの両方でコンパイルエラーになります。実質的な型付き API クライアントとして機能しています。

12 テスト

ファイル：`src/lib/date.test.ts`。テスト対象の選び方とケースの選び方に、はっきりした方針があります。

12-1. なぜ `date.ts` だけをテストしているのか

`date.ts` は純粋関数（pure function）の集まりだからです。

- 引数だけで結果が決まる（DB もネットワークも触らない）
- 副作用がない
- モックが一切不要

一方 Service や Route は DB やセッションに依存するので、テストにはモックやテスト用 DB の用意が必要です。

テストの費用対効果

「テストの費用対効果が最も高い純粋ロジックから書く」のは正しい優先順位です。そして `getWeeks` はこのアプリで最も複雑でバグしやすいロジックなので、まさにテストすべき対象です。

12-2. テストケースの選び方

テスト	検証している性質
<code>toStr(new Date(2026,0,5)) → "2026-01-05"</code>	ゼロ埋めと月の +1
<code>getWeeks(2026, 7).length === 5</code>	週の分割数
<code>getWeeks(2026, 8)[0][0] === null</code>	エッジケース：月初が月曜でない場合の穴埋め
すべての週がちょうど5要素	不変条件（invariant）
<code>dateToDateStr(dateStrToDate(x)) === x</code>	ラウンドトリップ

特に3つ目がエッジケース（2026年9月1日は火曜なので、最初の週の月曜が null になる）、4つ目が不変条件テスト（個別の値ではなく「常に成り立つ性質」を検証）です。この選び方は良質です。

12-3. AAA / BDD スタイル

`describe`（何をテストするか）→ `it`（どういう振る舞いか）→ `expect`（何を期待するか）という BDD スタイルです。テスト名が日本語で書かれているので、テスト結果がそのまま仕様書として読めます。

12-4. vitest.config.ts の設定

```
resolve: { tsconfigPaths: true }
```

テストファイル内の `@/lib/date` というパスエイリアスを Vitest に解決させる設定です。これがないと import が失敗します。

13 ツール・設定まわり

ファイル	役割
<code>tsconfig.json</code>	<code>"strict": true</code> で null チェック等を全部有効化。 <code>paths: { "@/*": ["/src/*"] }</code> で <code>../../../lib/x</code> を <code>@/lib/x</code> にするパスエイリアス
<code>eslint.config.mjs</code>	Flat Config 形式。バグになりうる書き方を静的検出
<code>.prettierrc</code>	フォーマット統一。 <code>prettier-plugin-tailwindcss</code> がTailwind クラスを公式推奨順に自動並べ替える
<code>postcss.config.mjs</code>	Tailwind v4 を PostCSS プラグインとして読み込む
<code>prisma.config.ts</code>	Prisma 7 の新しい設定ファイル。 <code>dotenv/config</code> で <code>.env</code> を読む
<code>vitest.config.ts</code>	パスエイリアスの解決設定
<code>next.config.ts</code>	現状は空。設定は TypeScript で型付きで書ける

13-1. postinstall: prisma generate

`npm install` 後に自動で Prisma Client を生成します。これにより、生成物を Git にコミットしない運用が可能になります。デプロイ先でも `npm ci` するだけで型が揃います。

13-2. output = "../../src/generated/prisma"

Prisma 7 では生成先を `node_modules` ではなく明示的なパスに出すのが標準になりました。`node_modules` に隠れないので、生成された型をエディタで直接読めます。ただし `src` 配下なので、`.gitignore` の扱いには注意が必要な部分です。

13-3. npm scripts

コマンド	内容
<code>npm run dev</code>	開発サーバー起動
<code>npm run build</code>	本番ビルド
<code>npm run typecheck</code>	<code>tsc --noEmit</code> 。型エラーだけを検査
<code>npm run lint</code>	ESLint
<code>npm run format</code>	Prettier で <code>src</code> を整形
<code>npm test</code>	Vitest（ウォッチモード）
<code>npm run test:run</code>	Vitest（1回だけ実行。CI 向け）

`test` と `test:run` を分けているのがポイントです。ウォッチモードは CI で無限に止まらなくなるため、CI では `test:run` を使います。

14 未完成部分・改善点

説明する際、ここを自分から挙げられると理解の深さと誠実さが伝わります。「知らない」のではなく「認識した上で優先順位をつけている」ことを示せます。

項目	内容
Attendance が UI につながっていない	<code>/api/attendances</code> と <code>getMonthlyAttendances</code> は完成しているが、 <code>Calendar.tsx</code> は日付を描画するだけで予定を表示していない。次の実装ステップはここ
Attendance の書き込み API がない	GET のみ。予定を登録するには POST / PUT (<code>upsert</code>) が必要。 <code>@unique([date, childId])</code> はその準備
タイムゾーンの不整合	第6章 6-4 で述べた <code>toStr</code> / <code>dateToDateStr</code> の混在。JST でのみ正しく動く状態
<code>page.tsx</code> のデッドコード	関数末尾の <code>redirect("/login")</code> は <code>return</code> の後ろにあり絶対に実行されない。削除すべき
<code>Counter.tsx</code> が未使用	学習用の残骸。削除候補
<code>?limit=100</code> のハードコード	<code>useChildList.ts</code> がページネーションを使っていない。API 側は対応済みなので、児童数が増えたら繋ぐ
<code>approved</code> の管理画面がない	現状は DB を直接操作する必要がある。管理者用 UI が必要
<code>defaultTimeFrame</code> が未使用	スキーマにはあるがロジックで参照されていない
<code>globals.css</code> の font-family	<code>next/font</code> で Geist を読み込んでいるのに、body で <code>Arial</code> を直接指定して上書きしている。 <code>font-sans</code> を使うべき
ファイル名タイポ	<code>ChildListSkelton.tsx</code> → <code>Skeleton</code>
Child の更新・削除 API	現状 POST (作成) と GET (一覧) のみ。PATCH / DELETE が未実装

14-1. 優先順位をつけるなら

- Attendance の POST (upsert) + Calendar への表示 — アプリの主目的そのもの。ここが動いて初めて「予定表」になる
- タイムゾーンの統一 — 日付アプリの根幹。潜在バグとして最も危険
- `approved` の管理 UI — 運用に必須
- デッドコード・タイポの整理 — コストが低く、すぐ終わる
- ページネーションの接続 — 児童数が増えてから

15 まとめ

15-1. この設計を一言で説明するなら

「型と制約を上流に置き、下流を安全にする」設計。

- DB のユニーク制約が重複を防ぎ、
- Zod スキーマ1つが型・クライアント検証・サーバー検証の3役をこなし、
- レイヤー分割で各層の責務を限定し、
- React Query の queryKey がコンポーネント間を疎結合につなぎ、
- URL がアプリの状態を持つことで状態管理コードを不要にしている。

説明を求められたときは、この5行を軸に「どの部分の話ですか？」と展開していけば筋が通ります。

15-2. 貫いている3つの原則

▶ 原則1：チェックはできるだけ上流（外側）で行う

DB の制約 → Zod スキーマ → 型システム、という順に「後から破れない場所」に条件を置いています。アプリのコードで `if` を書くのは最後の手段です。

▶ 原則2：1つの定義から複数の用途を導出する

- Prisma スキーマ → DB テーブル + TypeScript 型 + マイグレーション
- Zod スキーマ → 実行時検証 + TypeScript 型 + エラーメッセージ

手で二重管理する箇所を徹底的に減らしています。

▶ 原則3：知らなくていいことは知らせない

- Service は HTTP を知らない
- Repository は業務ルールを知らない
- ChildForm は ChildList を知らない（queryKey だけで繋がる）
- API は DB のカラム構成を外に出さない（DTO）

15-3. 説明の難所と、その乗り切り方

難所	説明の切り口
<code>getWeeks</code> の週分割	「その日が属する週の月曜日を計算して、それをグルーピングのキーにしている」の1文から始める。あとは「なぜ null で5マス埋めるか＝曜日とインデックスを対応させるため」を足す
React Query のキャッシュ無効化	「ChildForm と ChildList は互いを知らない。queryKey という共通の名前だけで繋がっている」と言い切る
Server / Client 境界	「import した子は Client になるが、children で渡した子はならない」を先に言う
Zod と型の関係	「TypeScript の型はビルド時に消えるので、外から来るデータは実行時に検証しないと意味がない」
なぜ層を分けるのか	「Service は Request を受け取らないので、テストで HTTP を用意しなくても呼べる」という具体的な利益で答える
401 と 403	「401 はあなたが誰か分からない、403 は誰かは分かるが権限がない」

— 本書は schedule-next のソースコード全ファイルに基づいて作成されています —